

Explaining the Monte Carlo Script

A Walkthrough of scripts/spurious_stationary_sim.R

Eliab Luvanda

2026-08-09

Table of contents

1	How to use this document	1
2	What the script computes	2
3	Seven design decisions	2
4	R idioms used in the script	3
5	Setup and helper functions	3
5.1	Global options (lines 19–39)	3
5.2	Drawing series (lines 47–73)	4
5.3	HAC estimation (lines 75–106)	4
5.4	The three test statistics (lines 112–139)	5
6	The main experiment	5
6.1	simulate_dgp (lines 220–257)	5
6.2	Benchmarks and comparison tables (lines 259–289)	5
7	The four extension experiments	6
7.1	Common stochastic volatility (lines 299–340)	6
7.2	Local-to-unity sequences (lines 353–378)	6
7.3	Distributed lags and Granger causality (lines 401–465)	6
7.4	Local power (lines 482–542)	6
8	The accessor layer	7
9	Modifying the script	8
10	Outputs, runtime and reproducibility	8
11	What the script does not do	9

1 How to use this document

This document walks through scripts/spurious_stationary_sim.R, the 767-line script that produces every figure, table, and quoted number in the handout *Is Spurious Regression a Problem in Stationary Time Series?* It depends only on ggplot2 and scales.

Its two audiences want different things from this walkthrough.

If your interest is **applied**, you need only enough to trust the evidence and re-run it yourself. Read Sections 2 and 3, skim the walkthrough for the experiments you care about, and stop — you do not need to be able to write this code to rely on it.

If your interest is **theoretical**, or you plan to teach this material, the script is meant to be modified, not just run. The single most useful exercise the handout suggests is deriving λ for a process of your own and confirming it numerically. Section 9 walks through exactly how, and the script is organised so that this is a ten-line change, not a rewrite.

A note on line numbers. Citations below refer to the script as of this writing and will drift as soon as anyone edits it. The script's own banner comments — # 1. Primitive helpers, # 2. Test statistics, and so on — are the stable landmarks; this walkthrough follows them.

2 What the script computes

Everything in the script is ultimately one of two calculations, repeated many times under different conditions.

A slope test. Regress y on a constant and x and ask whether the slope differs significantly from zero. The data are always generated with a true slope of zero, so every rejection is a false positive; counting rejections across replications estimates the test's size, which should be 5% if the test is working.

A joint test on several lags. Regress y on its own past and on lags of x , and ask whether the x lags jointly matter. Again the truth is that they do not, so rejections are false positives.

Both are computed with three kinds of standard error — conventional, heteroskedasticity-consistent (HC0), and heteroskedasticity-and-autocorrelation-consistent (HAC) — so the methods can be compared on identical data.

There are four experiments beyond the main grid: common stochastic volatility, local-to-unity sequences, distributed-lag Granger tests, and local power. Section 7 covers each.

3 Seven design decisions

Most of the script's structure follows from seven choices. Understanding them makes the code far easier to read than working through it line by line.

1. One source of truth. The handout contains no hard-coded numbers. Every figure quoted in its prose is read from this script's output at render time, through the accessor functions in Section 9. Text and simulation therefore cannot silently disagree — the most common defect in simulation-based papers — and changing a parameter here updates the handout's prose on the next render.

2. One HAC estimator everywhere. An earlier version used a hand-written Bartlett estimator for the slope tests and the sandwich package for the Wald tests — two different small-sample corrections, making the tables not strictly comparable. The current script implements both the scalar and matrix versions itself (`hac_lrv` and `hac_lrv_matrix`), applies the same degrees-of-freedom adjustment to both, and depends on `sandwich` not at all.

3. Data-generating processes live in a registry, not an if chain. Each process is an entry in a list carrying a name, a type, its theoretical λ , and a function that draws a sample. The simulation engine never asks which process it is running, so adding a new one never touches the engine.

4. Every experiment is seeded independently. A single `set.seed` at the top of a script is a trap: extension experiments would inherit whatever random state the main loop happened to leave behind, so changing the replication count in one experiment would silently change the results of all the others. The script instead uses a named vector of five seeds, one per experiment.

5. Accessors fail loudly. The functions the handout calls to retrieve numbers stop with an error if they match anything other than exactly one record. A typo in a process name produces a failed render, not a plausible-looking wrong number.

6. Plot limits never delete data. `scale_y_continuous(limits = ...)` in `ggplot2` *removes* observations outside the range rather than clipping the view. The script uses `coord_cartesian()` and `expand_limits()` instead, which zoom without discarding.

7. A quick mode for drafting. Setting `SPURIOUS_QUICK=1` reduces every replication count to 200 — noisy but structurally identical, and well under a minute instead of the full run's twenty to forty. Use it while editing; do one full run before circulating anything.

4 R idioms used in the script

The script is written in base R with `ggplot2` for graphics. A few constructions recur and may be unfamiliar if your R is mostly `lm()` and `read.csv()`.

`function(n) list(x = ..., y = ...)` A function stored in a variable and called later. The DGP registry stores one of these per process.

`force(phi_x)` **inside** `ar_pair()` R evaluates arguments lazily, on first use. Building a list of functions in a loop without forcing them is a classic bug: every function ends up using the *last* set of parameter values, because none was evaluated until the loop finished. `force()` evaluates the argument immediately and pins it — why `ar_pair(0.5, 0.5)` and `ar_pair(0.8, 0.8)` stay distinct.

`stats::filter(..., method = "recursive")` Generates an autoregressive series in compiled code — far faster than an R loop.

`stats::filter(..., method = "convolution", sides = 1)` Generates a moving-average series. The first observations come back NA because the filter needs prior values that do not exist yet; the script discards them.

`vapply(x, f, numeric(1))` Like `sapply`, but with a declared return type, so an unexpected result errors immediately instead of silently producing a list.

`do.call(rbind, list_of_data_frames)` Stacks a list of data frames into one — the idiom for assembling results collected in a loop.

`array(..., dim = c(a, b, c))` A three-dimensional table. The script uses these to hold results indexed by sample size, method, and replication simultaneously.

`.one(v, what)` Not standard R — a helper defined in the script. The leading dot is the usual convention for an internal helper not meant to be called directly.

5 Setup and helper functions

5.1 Global options (lines 19–39)

Sample sizes, replication counts, the significance level, and the five seeds. Replication counts differ by experiment — 2000 for the main grid, the volatility experiment, and power; 1000 for local-to-unity and Granger. Fewer replications mean noisier estimates, so the handout reports the implied Monte Carlo standard error alongside its tables, letting readers judge which differences are real.

5.2 Drawing series (lines 47–73)

```
draw_ar1 <- function(n, phi) {
  as.vector(stats::filter(
    rnorm(n), phi, method = "recursive",
    init = rnorm(1, sd = 1 / sqrt(1 - phi^2))
  ))
}
```

The `init` argument is the part that is easy to get wrong. An AR(1) started at zero is not stationary at the beginning of the sample — its variance grows from zero toward the stationary value, and with $\phi = 0.95$ that takes hundreds of observations. Any rejection frequency computed over that stretch would confound the effect under study with a start-up artefact.

Drawing the pre-sample value from $N\{0, 1/(1 - \phi^2)\}$ — the process’s own long-run distribution — makes the first observation stationary already, so no burn-in is needed and no observation is wasted. The alternative, generating extra observations and discarding them, is slower and only approximate.

`draw_ma2` needs no such care, because a moving average is stationary from its first observation by construction.

5.3 HAC estimation (lines 75–106)

`hac_bandwidth(n)` returns $\lfloor 2n^{1/3} \rfloor$. The exponent $1/3$ is the mean-squared-error-optimal rate for the Bartlett kernel; the multiplier of 2 is a deliberate choice, discussed in the handout, that is considerably more generous than the common Newey–West rule of thumb — 34 lags at $T = 5000$ where Newey–West gives 9. Persistent processes need the extra lags, and the handout shows that even 34 is not enough at $\phi = 0.95$.

`hac_lrv` implements the Bartlett estimator directly:

```
hac_lrv <- function(score, L, dof_scale = 1) {
  n <- length(score)
  v <- sum(score^2) / n
  if (L > 0) {
    for (lag in seq_len(L)) {
      g <- sum(score[(lag + 1):n] * score[1:(n - lag)]) / n
      v <- v + 2 * (1 - lag / (L + 1)) * g
    }
  }
  v * dof_scale
}
```

Read this against the decomposition in the handout: the line before the loop is the contemporaneous term that produces κ , and the loop adds the serial terms, each weighted by $1 - \text{lag}/(L + 1)$ — a weight that declines linearly to zero. That decline is not cosmetic; it is what guarantees the estimate cannot come out negative, which an unweighted sum can.

`hac_lrv_matrix` is the same calculation for a vector-valued score, needed for the joint Wald tests. `crossprod(S)` computes $S'S$ efficiently, and the loop adds $G + G'$ at each lag to keep the result symmetric.

5.4 The three test statistics (lines 112–139)

slope_t_stats computes the same slope estimate three times with three different standard errors:

```
se_conventional <- sqrt(sum(resid^2) / (n - 2) / sxx)
se_hc0 <- sqrt(sum(score^2) * dof / sxx^2)
se_hac <- sqrt(n * hac_lrv(score, hac_bandwidth(n), dof) / sxx^2)
```

These three lines are the handout’s whole argument in miniature. The conventional formula multiplies two separately estimated variances. HC0 uses the actual products $x_t \hat{u}_t$, so it survives large regressor values coinciding with large errors. HAC additionally sums those products across time, so it survives serial correlation too. Each is strictly more general than the one above it, and each pays for that generality with extra estimation noise — which is why the handout finds HC0 beating HAC in the volatility experiment, where the extra generality buys nothing.

6 The main experiment

6.1 simulate_dgp (lines 220–257)

The engine. Each replication draws one series of length 5000 and evaluates the test on every prefix — $T = 50, 100, 150, \dots$. Nesting the samples this way keeps the curves across T smooth enough to read, since adjacent points share most of their data.

The prefix calculation uses running sums rather than refitting the regression 100 times:

```
cs <- cbind(cumsum(x), cumsum(y), cumsum(x^2), cumsum(y^2), cumsum(x * y))
```

Once these five cumulative sums exist, the regression at any sample size is arithmetic away: row n gives the sums for the first n observations, and the centred quantities follow from the standard identities. The residual sum of squares uses a shortcut worth knowing:

$$SSR = S_{yy}^c - \hat{\beta} S_{xy}^c,$$

which holds because $\hat{\beta} S_{xx}^c = S_{xy}^c$ by definition of the slope. This turns 100 regressions into five cumulative sums and some vector arithmetic, and it is the reason the main experiment finishes in minutes rather than hours.

The HAC statistic cannot be computed this way, because a long-run variance is not a running sum. It is evaluated only at six selected sample sizes instead — why the HAC figures in the handout show points at $T \in \{50, 200, 500, 1000, 2000, 5000\}$ while the conventional figure shows a continuous curve.

6.2 Benchmarks and comparison tables (lines 259–289)

After the loop, res and hac_res hold the rejection frequencies. The script attaches the Monte Carlo standard error of each proportion, then merges in the theoretical λ and the implied asymptotic rejection probability

$$2\Phi\left(-\frac{1.96}{\sqrt{\lambda}}\right)$$

so simulated and theoretical values sit side by side in the handout’s tables. This is the script’s own check on itself: if the simulated frequencies did not converge toward the theoretical column, something would be wrong in the theory or the code.

7 The four extension experiments

7.1 Common stochastic volatility (lines 299–340)

Both series are built as a shared volatility path multiplied by independent draws:

```
s <- exp(h / 2)
x <- s * rnorm(Tmax)
y <- s * rnorm(Tmax)
```

Because the two `rnorm` draws are independent, the population slope is exactly zero. Because both are scaled by the same s , the series are dependent — calm together, wild together. This is the handout’s counterexample to the belief that a serially uncorrelated score implies correct size: here the score has no serial correlation at all, and the conventional test still rejects about 12.7% of the time.

The parameter controlling the distortion is `volatility_variance`, the variance of $h_t = \log s_t^2$. Theory gives $\lambda = e^{v_h}$ exactly, so setting it to 0.5 predicts $\lambda \approx 1.65$ — change this one number and rerun to watch the theory track the simulation.

7.2 Local-to-unity sequences (lines 353–378)

Here ϕ depends on the sample size, $\phi_T = 1 - 5/T$. This is a *triangular array*: the data-generating process differs at every T , so unlike the main experiment there is no single underlying process being sampled ever more finely.

One consequence deserves emphasis, because it is a modelling choice rather than a technicality: the pre-sample value is drawn from the invariant distribution, as everywhere else in the script. With $\phi = 0.999$ that distribution has standard deviation about 22, so the series begins far from zero. This is the *stationary-start* design; a zero-start design gives materially different finite-sample numbers. Neither is more correct — they answer slightly different questions, and the handout says which one it uses.

7.3 Distributed lags and Granger causality (lines 401–465)

Two regressions are fitted to the same data: one including y_{t-1} , one omitting it. Both are tested for the joint significance of two lags of x .

The design deserves care in interpretation, and the script’s comment header says so explicitly. Because x and y are generated independently, the omitted y_{t-1} is uncorrelated with the included x lags, so the population coefficients on those lags are zero in *both* specifications — there is no omitted-variable bias anywhere in the experiment. Everything the misspecified column shows, a true null rejected about a third of the time, comes purely from a mismeasured standard error. That is why HAC repairs most of it, and why the experiment must not be read as evidence that HAC fixes omitted-variable bias.

`wald_reject` returns NA rather than failing when the covariance matrix is numerically singular, which can happen for HAC at the smallest sample sizes; those replications are dropped from the average via `na.rm = TRUE`.

7.4 Local power (lines 482–542)

The newest experiment, and the most subtle. Data are generated under a *local alternative*,

$$y_t = \frac{b}{\sqrt{T}}x_t + v_t,$$

where the true slope shrinks as the sample grows. This is standard practice: with a fixed non-zero slope, every reasonable test's power approaches one and the comparison becomes uninformative. Shrinking the alternative at rate $1/\sqrt{T}$ keeps the problem hard at every sample size.

The critical step is **size adjustment**. Each test is compared not with 1.96 but with the empirical 95th percentile of its own $|t|$ statistics under $b = 0$:

```
null_index <- which(b_grid == 0)
cv <- vapply(methods, function(m)
  unname(quantile(abs(tstat[, null_index, m]), 1 - alpha)), numeric(1))
```

Without this step the comparison is meaningless: a test that rejects 35% of the time when nothing is happening will of course reject more often when something is. Size adjustment asks the only fair question — if both tests were correctly calibrated, which detects the alternative more often? The handout's answer is that they are close: HAC's power cost, properly measured, is much smaller than the folklore suggests.

The size-adjusted critical values are reported for their own sake too, because they test the handout's central proposition directly. Theory says the conventional statistic has spread $\sqrt{\lambda}$ rather than 1, so its 5% critical value should converge to $1.96\sqrt{\lambda}$. For the $\phi = 0.8$ pair, $\lambda = 4.56$ and that product is 4.18; a simulated value near 4.18 that does not shrink as T grows confirms the proposition far more sharply than any rejection frequency could.

8 The accessor layer

Section 9 of the script exists entirely to serve the handout. It converts stored results into formatted strings:

```
rp("(0.8,0.8)", 5000)          # simulated rejection rate, e.g. "34.8%"
asy_of("(0.8,0.8)")           # the theoretical benchmark for comparison
gr_rej("Omit lagged y", "HAC", 2000)
pw_cv("Conventional OLS", 250) # the size-adjusted critical value
```

In the handout these appear inside inline R, so the sentence “the conventional test rejects `rp("(0.8,0.8)", 5000)` of the time” prints whatever the current simulation produced.

Each is a thin wrapper around `.one`, which enforces that exactly one record matched:

```
.one <- function(v, what) {
  v <- v[!is.na(v)]
  if (length(v) != 1L)
    stop(sprintf("accessor '%s' matched %d records, expected 1", what, length(v)))
  v
}
```

This is defensive by design. Without it, a mistyped process name would return an empty vector, and the handout would render with a blank where a number should be — or worse, a silently wrong value averaged over several records. With it, the render fails and names the offending call.

9 Modifying the script

The handout suggests deriving λ for a process of your own and confirming it numerically. Here is the whole procedure.

Step 1: work out λ on paper. For independent linear processes,

$$\lambda = \sum_{k=-\infty}^{\infty} \rho_x(k) \rho_y(k),$$

the lag-by-lag product of the two autocorrelation functions.

Step 2: add an entry to the registry. In Section 3 of the script, add to `dgp_registry`:

```
list(dgp = "my process", type = "stationary",
     lambda = <your value>,
     draw = function(n) list(x = <draw x>, y = <draw y>))
```

Step 3: give it a colour. Add an entry to `palette` in Section 10 keyed by the same name. The script uses named colours, so an omission produces a missing series rather than a wrong one.

Step 4: run. Nothing else needs to change — the engine, the tables, the figures, and the accessors all pick it up automatically. Use `SPURIOUS_QUICK=1` for the first attempt.

Step 5: check. Compare the simulated rejection frequency at $T = 5000$ with the asymptotic column the script computes from your `lambda`. Disagreement by more than about two Monte Carlo standard errors means either the derivation or the draw function is wrong — a genuine test that does catch mistakes.

Two other modifications are worth trying. Replacing `hac_bandwidth` with the Newey–West rule of thumb, `floor(4 * (n/100)^(2/9))`, shows how much the HAC results depend on a tuning choice that is rarely reported. And changing `volatility_variance` in the common-volatility experiment moves $\lambda = e^{\nu_h}$ in a way you can predict exactly before running it.

10 Outputs, runtime and reproducibility

Running

```
Rscript scripts/spurious_stationary_sim.R
```

writes six PDF figures, nine CSV result files, and a `sessionInfo.txt` recording the R and package versions used. The output directory defaults to `scripts/` and can be redirected with the `SPURIOUS_OUTPUT_DIR` environment variable.

The script deliberately never `print()`s its plots — that would create a stray `Rplots.pdf` every time the handout sources it, a small but persistent annoyance. The plot objects `p1` through `p6` are simply left in the environment for the handout to render.

Sourcing from the handout is cached on a hash of the script file. The handout’s setup chunk, named `simulation-data`, carries `include=FALSE`, `cache=TRUE`, and `cache.extra=tools::md5sum("scripts/spurious_stat` with the single line `source("scripts/spurious_stationary_sim.R")` as its body. That third option does the work: it makes the cache key depend on the script’s contents, so the simulation reruns when and only when the script changes. Editing the handout’s prose does not trigger a lengthy recomputation.

11 What the script does not do

Three honest limitations, worth stating so that nobody infers more from the evidence than it supports.

Only Gaussian innovations. Every process is driven by normal draws. The theory needs only finite fourth moments, so heavier-tailed innovations would be a reasonable robustness check — one this script does not run.

Only the Bartlett kernel, at one bandwidth rule. No quadratic-spectral kernel, no prewhitening, no data-dependent bandwidth, no fixed- b critical values. The handout identifies all of these as natural remedies where HAC struggles and cites the relevant literature, but implements none of them. Extending the script in this direction would be a substantial and worthwhile project.

Size, and one power experiment. The main grid measures size only; power is measured for a single configuration, the $\phi = 0.8$ pair at two sample sizes. Broader power comparisons across the DGP grid would be a natural extension, and would reuse `simulate_local_power` almost unchanged.